Typst w pigułce

składnia, matematyka, skrypty, kod i wykresy w jednym dokumencie

1. Wstęp

Ten dokument pokazuje możliwości Typsta na własnym przykładzie: powstał z jednego pliku tekstowego (`typst_w_pigulce.typ`) i jednego skryptu Pythona (`generuj_wykres.py`). Typst jest językiem składu tekstu podobnym do LaTeX-a, ale kompiluje się w ułamku sekundy, a jego część skryptowa jest prawdziwym językiem programowania. Ten egzemplarz, w stylu **artykuł**, ma 4 strony, a liczbę tę wstawił sam Typst.

W kolejnych sekcjach pokazujemy po kolei składnię, matematykę, kod wczytywany z pliku, obraz wygenerowany przez Pythona oraz obliczenia wykonywane w trakcie składania dokumentu. Na końcu wyjaśniamy, jak z tej samej treści powstają dokumenty o różnym wyglądzie.

2. Składnia

Zwykły tekst piszemy tak, jak jest, a formatowanie zaznaczamy prostymi znakami. W przykładach poniżej po lewej stoi to, co wpisujemy, a po prawej to, co z tego powstaje. Prawa strona nie jest obrazkiem: dokument wykonuje kod na żywo.

```
*pogrubienie*, _kursywa_, `kod`,
#underline[podkreślenie]
i #text(fill: red)[kolorowy tekst].
```

pogrubienie, *kursywa*, `kod`, podkreślenie i kolorowy tekst.

Listy numerowane i punktowane zaczynamy od `+` i `-`, a wcięcie tworzy zagnieżdżenie:

```
+ pierwszy punkt
+ drugi punkt
  - zagnieżdżona lista
+ trzeci punkt
```

1. pierwszy punkt
2. drugi punkt
   - zagnieżdżona lista
3. trzeci punkt

Odnośniki i przypisy też są zwykłymi poleceniami. Przypisy numerują się same i trafiają na dół strony:

```
Odnośnik: #link("https://typst.app")
[typst.app]. \
Przypis#footnote[Przypisy numerują się
same.] jest na dole strony.
```

Odnośnik: typst.app.
Przypis[1] jest na dole strony.

3. Matematyka

Wzory wpisujemy między znakami dolara. Zapis wygląda jak zwykły tekst, bez ukośników i klamer. Gdy po otwarciu i przed zamknięciem dolara stoi spacja, wzór wyświetla się w osobnej linii:

[1] Przypisy numerują się same.

```
$ sum_(k=1)^n k = (n (n+1)) / 2 $
```

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Wzory można numerować i się do nich odwoływać. Drgania tłumione, które narysujemy w dalszej części dokumentu, opisuje wzór 1:

$$y(t) = A e^{-t/\tau} \sin(\omega t) \tag{1}$$

Tu A oznacza amplitudę, τ stałą czasową, a ω pulsację. Pole pod obwiednią tych drgań wynosi

$$\int_0^\infty A e^{-t/\tau}\,\mathrm{d}t = A\tau. \tag{2}$$

4. Kod z pliku zewnętrznego

Funkcja `read` wczytuje dowolny plik tekstowy, a `raw` formatuje go jako kod z kolorowaniem składni. Poniższy skrypt Pythona nie jest wklejony do dokumentu, tylko wczytany z pliku `generuj_wykres.py`. Dzięki temu kod w PDF-ie zawsze zgadza się z tym, który faktycznie narysował wykres:

```python
"""Rysuje wykres drgań tłumionych do pliku wykres.png (obok tego skryptu).

Plik jest wczytywany do dokumentu typst_w_pigulce.typ jako zwykły tekst,
a wygenerowany przez niego obraz jest w tym dokumencie osadzony.
"""
from pathlib import Path

import matplotlib

matplotlib.use("Agg")  # bez okna: rysujemy wyłącznie do pliku
import matplotlib.pyplot as plt
import numpy as np

A, TAU, OMEGA = 1.0, 3.0, 2.0  # amplituda, stała czasowa, pulsacja

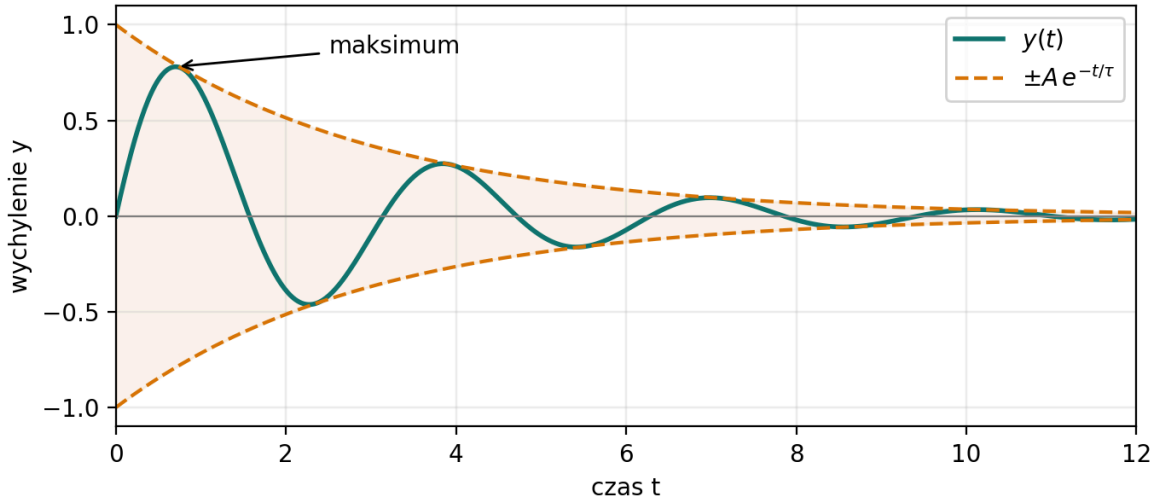
t = np.linspace(0, 12, 600)
obwiednia = A * np.exp(-t / TAU)
y = obwiednia * np.sin(OMEGA * t)

figure, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(t, y, color="#0f766e", linewidth=2, label=r"$y(t)$")
ax.plot(t, obwiednia, "--", color="#d97706", label=r"$\pm A\,e^{-t/\tau}$")
ax.plot(t, -obwiednia, "--", color="#d97706")
ax.fill_between(t, -obwiednia, obwiednia, color="#d97706", alpha=0.08)
ax.axhline(0, color="gray", linewidth=0.8)
ax.annotate(
    "maksimum",
    xy=(t[np.argmax(y)], y.max()),
    xytext=(2.5, 0.85),
    arrowprops={"arrowstyle": "→"},
)
ax.set_xlabel("czas t")
ax.set_ylabel("wychylenie y")
ax.set_xlim(0, 12)
ax.legend(loc="upper right")
ax.grid(alpha=0.25)
figure.tight_layout()

figure.savefig(Path(__file__).with_name("wykres.png"), dpi=200)
```

5. Obraz wygenerowany przez Pythona

Ten sam skrypt zapisał plik `wykres.png`, a Typst wstawia go poleceniem `image`. Rysunek 1 przedstawia przebieg opisany wzorem 1 dla $A = 1$, $\tau = 3$ i $\omega = 2$.



Rysunek 1: Drgania tłumione narysowane w `matplotlib`, a osadzone w dokumencie Typst.

6. Obliczenia w trakcie składania

Typst potrafi liczyć. Tabela poniżej nie została wpisana ręcznie, tylko obliczona z tego samego wzoru, który narysował wykres. Zmiana parametru w kodzie zmienia całą tabelę:

```
#let r(t) = str(calc.round(y(t), digits: 3))
#table(
  columns: 4,
  align: center,
  [*t*], [*y(t)*], [*t*], [*y(t)*],
  ..for t in range(0, 4) { (str(t), r(t), str(t + 4), r(t + 4)) },
)
```

t	y(t)	t	y(t)
0	0	4	0.261
1	0.652	5	−0.103
2	−0.389	6	−0.073
3	−0.103	7	0.096

Pętle, warunki, funkcje, tablice i słowniki działają tak samo jak w innych językach programowania. Ten sam mechanizm (funkcja `eval`) napędza przykłady w całym dokumencie: kod zapisany jako tekst jest najpierw pokazany, a potem wykonany.

7. Jeden plik, kilka stylów

Treść tego dokumentu leży w jednym pliku, a wygląd w osobnych konfiguracjach: `_styl_artykul.typ` (kolorowy nagłówek i akcenty) oraz `_styl_latex.typ` (czcionka Computer Modern, wcięcia akapitów, czarno-biały skład jak w LaTeX-u). Każda konfiguracja jest modułem, który eksportuje funkcję `styl` i

kilka ustawień, a o wyborze decyduje parametr z wiersza poleceń. Fragment tego pliku, który to robi, wczytany z niego samego:

```
#let style = (artykul: artykul, latex: latex)
#let wybrany = sys.inputs.at("styl", default: "artykul")
#assert(wybrany in style, message: "nieznany styl: " + wybrany + " (dozwolone: " +
style.keys().join(", ") + ")")
#let s = style.at(wybrany)
```

Dzięki temu poprawka w treści trafia do wszystkich wersji naraz, a nowy wygląd to jeden dodatkowy plik.

8. Jak powstał ten PDF

Krok	Polecenie
1. Wykres	`python3 generuj_wykres.py` zapisuje `wykres.png`
2. Artykuł	`typst compile --input styl=artykul typst_w_pigulce.typ ...`
3. LaTeX	`typst compile --input styl=latex typst_w_pigulce.typ ...`

Pełna dokumentacja Typsta: typst.app/docs.